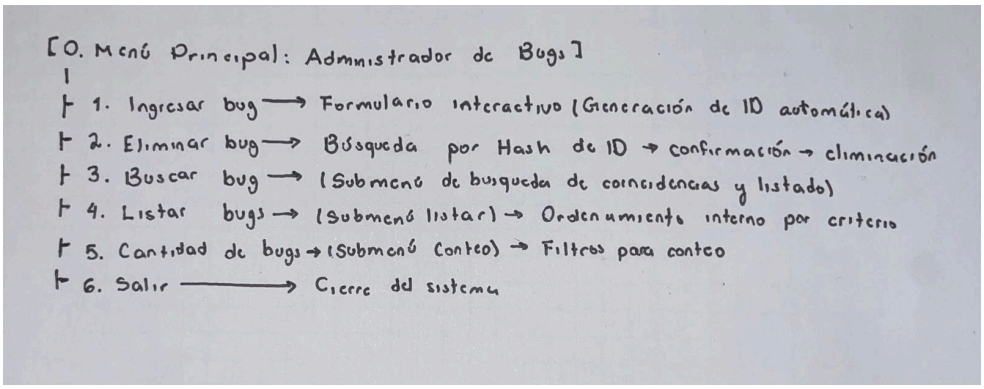
Gestión de Reportes de Bugs

Estructuras de Datos y Algoritmos II

Restricciones:

- No utilizar bibliotecas automáticas de ordenamiento o búsqueda (como Collections.sort() o Arrays.binarySearch()). Implementación manual obligatoria.
- No utilizar manejo de archivos ni hilos (Thread). Todo se gestiona en memoria principal (RAM).
- El usuario únicamente selecciona criterios de consulta, nunca el algoritmo de ejecución.
- El sistema debe iniciar con al menos 200 reportes de bugs ya cargados en memoria

1. Arquitectura de Navegación de la Consola



2. Especificación Detallada de Pantallas y Menús

---

0. Menú Principal

```
================================================
        ADMINISTRADOR DE REPORTES DE BUGS
================================================
Total de bugs activos en sistema: [N] (Inicial >= 200)


1. Ingresar bug
2. Eliminar bug
3. Buscar bug
4. Listar bugs (Ordenados)
5. Cantidad de bugs (Métricas / Filtros de conteo)
6. Salir
Seleccione una opción: _
```

Opción 1: Ingresar bug

Solicita los campos requeridos y calcula el ID de forma transparente mediante una función Hash matemática basada en título + fecha.

- **Título:** Cadena descriptiva del error.
- **Prioridad:** Entero (1 = baja, 2 = media, 3 = alta).
- **Nivel de seguridad:** Entero del 1 al 5.
- **Equipo encargado:** Cadena de texto (ej. "Backend", "Frontend", "Seguridad").
- **Categoría:** Tipo de bug (ej. "interfaz", "lógica", "rendimiento", "seguridad").
- **Estado:** 1 = abierto, 2 = en progreso, 3 = cerrado.
- **Fecha de reporte:** Formato YYYY-MM-DD.

Opción 2: Eliminar bug

- Solicita el ID del bug.
- Acceso y localización mediante la tabla Hash en tiempo esperado O(1).
- Muestra los datos del bug y solicita confirmación (Y/N) antes de removerlo de las estructuras en memoria.

Opción 3: Submenú "Buscar bug"

```
------------------------------------------------
                BUSCAR BUGS
------------------------------------------------
1. Título
2. ID
3. Fecha Reporte
4. Compuesto (Múltiples filtros combinados)
5. Mostrar todos sin filtro
6. Regresar al menú principal
Seleccione una opción: _
```

Comportamiento interno:

- **ID:** Búsqueda directa por Transformación de Claves (Hash con manejo de colisiones).
- **Título / Fecha:** Búsqueda Binaria (si la colección está ordenada por dicho criterio) o Búsqueda Lineal.
- **Compuesto:** Búsqueda Lineal multivariable para filtrar combinaciones (ej. categoría + prioridad).
- **Salida:** Despliega el número de coincidencias encontradas y el desglose de los reportes.

Opción 4: Submenú "Listar bugs" (Ordenamiento)

Criterio Elegido por el Usuario	Algoritmo Ejecutado Internamente	Familia Algorítmica	Complejidad Teórica
Prioridad (1-3) / Nivel de seguridad (1-5)	Counting Sort	Distribución	O(n + k)
Título / Equipo encargado / Categoría	QuickSort	Divide y vencerás	O(n log n) promedio
Fecha de reporte / ID	HeapSort o MergeSort	Divide y vencerás	O(n log n) garantizado

Opción 5: Submenú "Cantidad de bugs" (Métricas)

Permite contabilizar frecuencias por criterio o valores específicos (ej. desglose de cuántos bugs hay de prioridad 1, 2 y 3, o cuántos pertenecen a cada equipo encargado).

3. Especificaciones de la clase Bug

Atributos de instancia

- **id (String):** Identificador único del reporte. No es capturado por el usuario; se genera de forma automática aplicando la función hash sobre la combinación de título y fecha.
- **titulo (String):** Descripción del bug. Sirve tanto de contexto como de base para el cálculo del hash.
- **prioridad (int):** Nivel de prioridad de atención. Valores: 1 (Baja), 2 (Media) y 3 (Alta).
- **nivelSeguridad (int):** Nivel de vulnerabilidad del 1 al 5. 1 siendo muy vulnerable, 5 muy

seguro.

- **equipoEncargado (String):** Nombre del área asignada a la resolución (Frontend, Backend, Seguridad, Base de Datos).
- **categoria (String):** Clasificación del tipo de error (interfaz, lógica, rendimiento, seguridad).
- **estado (String):** Estado del reporte (recibido, en progreso, cerrado).
- **fechaReporte (String):** Fecha de captura en formato YYYY-MM-DD.

Métodos

1. Constructores

- **Constructor Principal:** Recibe los datos capturados o generados aleatoriamente (título, prioridad, nivel de seguridad, equipo, categoría, estado, fecha). En su interior, invoca a la función hash para asignar el id de forma automática.
- **Constructor Sobrecargado:** Recibe todos los atributos incluyendo un id predefinido (Para pruebas).

2. Getters y setters.

3. Función Hash

El id no es asignado por el usuario, sino generado automáticamente a partir de la concatenación del título descriptivo y la fecha de captura (titulo + fechaReporte). Para lograr una dispersión casi aleatoria, evitar colisiones por anagramas y garantizar claves uniformes. El método de cálculo del identificador opera en dos fases consecutivas:

Fase 1: Condensación del String por Ponderación Acumulada (Regla de Horner)
Para convertir el string en un valor entero sin provocar colisiones por transposición de caracteres, se evalúa la cadena como un polinomio donde cada carácter aporta su valor ASCII ponderado por la posición:

- **Fórmula polinómica:** Utiliza una constante *p = 31* (constante estándar empleada por el algoritmo de cadenas en Java) y módulo *M = 10^9 + 7* (pues es un número primo, el más pequeño justo después de 10^9, cifra fácil de recordar y escribir, también se

mantiene dentro de los límites seguros de un entero de 32 bits(2x(10^9))).

- **Aplicación de la Regla de Horner:** Optimiza la evaluación a tiempo lineal O(n):

  H = (H * 31 + c) mod (10^9 + 7).

  Con H = 0 al inicio.

Fase 2: Hashing de Fibonacci por desplazamiento de bits

Una vez condensada la cadena en el entero *k*, se realiza la dispersión al utilizar el producto de k y la constante *s* de 32 bits derivada de la Razón Áurea (2^32 / φ) redondeada para generar un valor cuyos primeros 10 bits (de izquierda a derecha) formarán el índice del bug dentro del contenedor:

- **Cálculo de la cubeta base:** Para mapear el valor a una tabla de tamaño *M = 1024* (2^10 cubetas), se multiplica *k* por *s* y se extraen los 10 bits más significativos mediante un desplazamiento a la derecha de 22 bits (32 - 10):

4. Formato de Despliegue en Consola: Sobrescritura de toString()

Por defecto, todo objeto en Java hereda de la clase base Object una implementación de toString() que devuelve únicamente el nombre canónico de la clase seguido del identificador hexadecimal de memoria (por ejemplo, modelo.Bug@6b884d06). Al listar o consultar colecciones de más de 200 registros en consola, esta salida resulta inutilizable. Sobrescribir el método toString() en la clase entidad Bug permite que cualquier instrucción de impresión (como System.out.println(bug) o al iterar estructuras lineales) proyecte inmediatamente una fila estructurada, limpia y autoalineada. Bae whatsApp answer

4. Generador de Bugs (GeneradorBugs)

Propósito y Restricciones Técnicas

- **Objetivo:** Poblar el sistema con una colección mínima de N >= 200 reportes de bugs realistas antes de cualquier interacción del usuario en el menú principal.

Estrategia para desarrollar la precarga con +200 bugs:

A. Generación aleatoria del Título mediante buckets de “errores”

Los títulos estarán conformados por 3 secciones, creadas con la idea de que al juntarse tengan total sentido.

- **1. Acción/Fallo:** {"NullPointerException al procesar", "Fallo de renderizado en", "Desbordamiento de memoria en", "Error 404 al consultar", "Bloqueo por timeout en", "Lectura corrupta en"}
- **2. Componente/Módulo:** {"módulo de login", "pasarela de pagos", "carrito de compras", "tabla de clientes", "generador de PDFs", "API REST de autenticación", "panel de administración"}
- **3. Condición o Contexto:** {"bajo alta concurrencia", "con caracteres especiales", "en dispositivos móviles", "al reiniciar el servidor", "tras actualización de dependencias", "en conexiones lentas"}

```
Títulos posibles = |Acción| x |Componente| x |Contexto|
```

Con un arreglo de 6 acciones, 7 componentes y 6 contextos se obtienen 6 x 7 x 6 = 252 títulos únicos y diferentes, teniendo 10 de cada uno es posible general 1000 títulos diferentes.

B. Generación de Fechas Cronológicas (fechaReporte)

- **Formato que tendrán las fechas:** YYYY-MM-DD.
- **Rango temporal:** Un intervalo de tiempo real con sentido, es decir, del 2020 al 2025 por ejemplo.

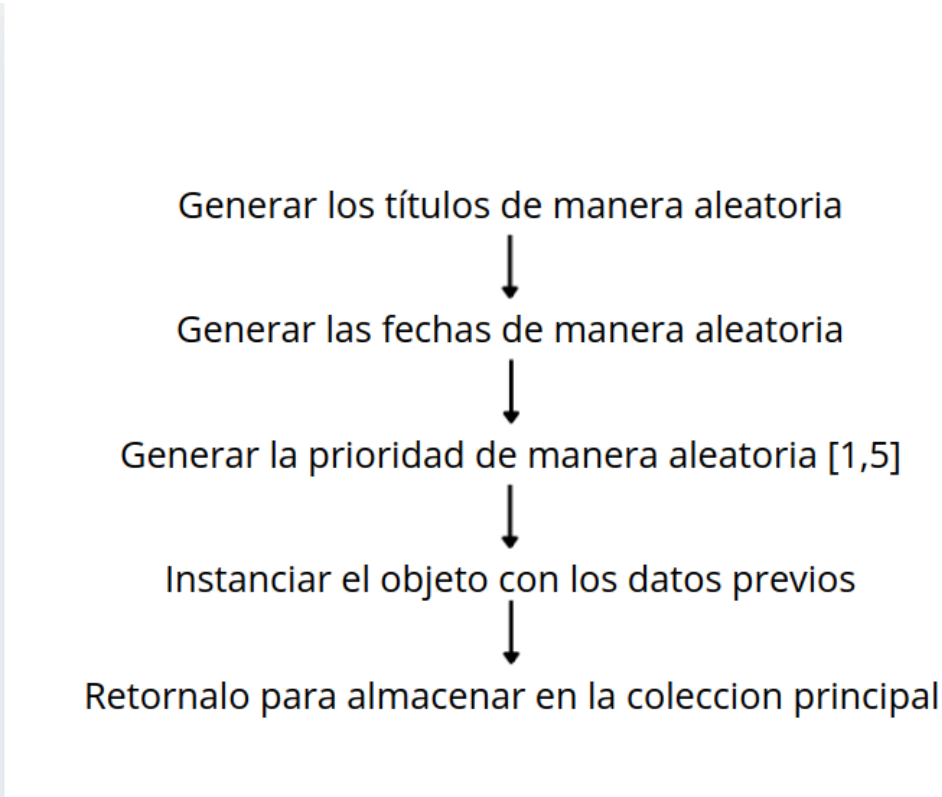
C. Atributos

- **Prioridad:** Entero pseudoaleatorio uniforme en el conjunto {1, 2, 3} (1 = baja, 2 = media, 3 = alta).
- **Nivel de Seguridad:** Entero pseudoaleatorio uniforme en el conjunto {1, 2, 3, 4, 5}.
- **Equipo Encargado:** También se seleccionará aleatoriamente de entre diferentes equipos, entre los que podrán estar: {"Frontend", "Backend", "BaseDatos", "Seguridad", "DevOps", "QA"}.
- **Categoría:** Selección aleatoria entre: {"interfaz", "lógica", "rendimiento", "seguridad",

"persistencia"}.

- **Estado:** Selección aleatoria del catálogo: {"abierto", "en progreso", "cerrado"}.

Flujo Algorítmico del Generador



Integración con el Repositorio de Datos (RepositorioBugs)

La clase solo se encarga de generar bugs de manera aleatoria, el método retorna un bug para que desde el almacén principal se guarde.

5. Almacén de Bugs (AlmacenBugs)

5.1. Propósito y Arquitectura en Memoria RAM

- **Rol en el sistema:** Actúa como el contenedor maestro o base de datos central en

memoria principal (RAM). Desacopla la lógica de almacenamiento respecto a los menús de la consola y a las bibliotecas de algoritmos.

- **Compromiso Espacio-Tiempo:** Cumpliendo con la restricción de no usar archivos ni excepciones, el repositorio gestiona una **estructura dual sincronizada** que comparte referencias a las mismas instancias de Bug:
    - **1. Estructura Lineal Contigua (Arreglo o Lista Dinámica):** Preserva los datos organizados secuencialmente para suministrarlos a los motores de ordenamiento interno (*QuickSort*, *HeapSort*, *Counting Sort*) y a las búsquedas comparativas (*Lineal* y *Binaria*).
    - **2. Estructura Hash Interna (Tabla Hash por Encadenamiento Separado):** Mapea los reportes a través de un arreglo primario de listas enlazadas/dinámicas (lista de listas). Permite resolver inserciones, eliminaciones físicas directas y búsquedas por clave en tiempo constante esperado O(1) sin requerir marcas de borrado lógico (centinelas).

5.2. Diseño de la Tabla Hash por Encadenamiento Separado

A. Extracción de la Clave Numérica

Como se estipuló en el diseño de la entidad, los reportes exponen un identificador formal formateado como "BUG-XXXXXX". La tabla hash descarta el prefijo constante "BUG-" y extrae exclusivamente el entero positivo de 6 dígitos $k \in [100\,000, 999\,999]$.

B. Función Hash de Cubeta (Mapeo a la Tabla)

- Para determinar el índice del cubo primario, se aplica el método de la división modular: `índice = k mod M`.
    - **Dimensión de la tabla (M):** Se define como un número primo adecuado (por ejemplo, M = 257 o M = 307). Esto distribuye uniformemente las claves entre los cubos.

C. Técnica de Resolución de Colisiones: Encadenamiento Separado (Lista de Listas)

En lugar de explorar casillas contiguas en el arreglo principal, cada posición del arreglo representa una cubeta que contiene una lista enlazada o lista dinámica secundaria:

- **Inserción y Mapeo:** Al calcular el índice de una clave k, el objeto Bug se añade

directamente a la lista secundaria ubicada en `tablaHash[índice]`.

- **Eliminación Física Directa:** La remoción de un elemento borra el nodo físicamente de la lista secundaria en `tablaHash[índice]`. Esto evita el uso de estados centinela o marcas "ELIMINADO" y previene el degradado en la eficiencia de exploración.

5.3. Atributos de la Clase

- **listaBugs (List<Bug>):** Lista lineal que conserva todos los reportes activos en memoria para operaciones globales de listado y ordenamiento.
- **tablaHash (List<List<Bug>>):** Lista principal de tamaño M inicializada con M sublistas internas independientes (ArrayList para acceso posicional inmediato a las cubetas, con sublistas LinkedList o ArrayList para encadenar colisiones).
- **capacidadHash (int):** Entero primo constante que define la cantidad de cubetas de la tabla (M).
- **totalBugs (int):** Contador de elementos activos presentes en el sistema (N ≥ 200).

5.4. Métodos y Operaciones Principales

Constructores de AlmacenBugs:

- **AlmacenBugs() / AlmacenBugs(int capacidad):** Asignan la capacidad prima M, instancian la colección secuencial `listaBugs` y pueblan cada una de las M cubetas de `tablaHash` con sublistas independientes vacías (`new LinkedList<>()` o `new ArrayList<>()`), garantizando que la estructura esté lista para recibir registros en tiempo constante desde su nacimiento.

Operaciones de Modificación y Consulta:

- **agregarBug(Bug nuevo) : boolean**
    - 1. Extrae la porción numérica del ID del bug.
    - 2. Calcula la cubeta correspondiente: `pos = idNumerico mod M`.
    - 3. Accede a la sublista en dicha posición (`tablaHash.get(pos)`) e inserta la referencia del objeto Bug.
    - 4. Inserta simultáneamente la referencia en la secuencia lineal `listaBugs`.

    - 5. Incrementa el contador `totalBugs`.
- **buscarPorId(String id) : Bug**
    - 1. Extrae el número del identificador y calcula `pos = idNumerico mod M`.
    - 2. Obtiene la sublista correspondiente a esa cubeta (`tablaHash.get(pos)`).
    - 3. Recorre únicamente los pocos elementos encadenados en esa cubeta comparando el id completo.
    - 4. Retorna la referencia al objeto Bug si coincide, o `null` si la sublista concluye sin hallarlo (tiempo promedio O(1 + α), donde α = N/M es el factor de carga promedio).
- **eliminarBugPorId(String id) : boolean**
    - 1. Localiza la cubeta mediante el residuo modular del ID.
    - 2. Recorre la sublista interna de esa cubeta; al encontrar el Bug, remueve el nodo de la sublista.
    - 3. Remueve la referencia correspondiente de la `listaBugs` lineal.
    - 4. Si se encontró y eliminó, decrementa `totalBugs` y retorna `true`; si no existía, retorna `false`.
- **obtenerLista() : List<Bug>**

Provee acceso a la colección secuencial para que las clases de ordenamiento y búsqueda comparativa puedan operar sobre los datos sin intervenir en el índice hash.

6. Módulo de Ordenamiento (ModuloOrdenamiento)

- Implementación de los algoritmos de ordenamiento de manera manual.
- El usuario selecciona el criterio de negocio desde un submenú (Prioridad, Nivel de seguridad, Título, Equipo, Categoría, Fecha, ID), pero es el sistema el que internamente ejecuta el algoritmo óptimo según el tipo de dato y su rango.
- El módulo integra al menos tres familias de ordenamiento interno:Distribución, Intercambio y Selección / Intercalación.

Criterios y Asignación de Algoritmos

- **Prioridad (1 a 3):** Entero acotado -> Counting Sort (Distribución)
- **Nivel de seguridad (1 a 5):** Entero acotado -> Counting Sort (Distribución)
- **Título:** String -> QuickSort (Intercambio)
- **Equipo encargado / Categoría:** String -> QuickSort (Intercambio)
- **Fecha de reporte (YYYY-MM-DD):** Cadena cronológica -> HeapSort (Max-Heap) (Selección)
- **ID (BUG-XXXXXX):** String -> HeapSort o MergeSort (Selección / Intercalación)

7. Módulo de Búsqueda (ModuloBusqueda)

Clasificación de Búsquedas y Criterios del Sistema

- **Búsqueda por ID:** Búsqueda Hash (Encadenamiento) - O(1)
- **Búsqueda por Título exacto o Fecha:** Búsqueda Binaria - O(log n)
- **Búsqueda por Categoría / Equipo / Estado:** Búsqueda Lineal Exhaustiva - O(n)
- **Búsqueda Compuesta:** Búsqueda Lineal Multivariable - O(n)
- **Cantidad de bugs por Criterio:** Búsqueda Lineal de Conteo / Reducción Binaria - O(n) o O(log n)

Detalles

- **1. Búsqueda Binaria de Extremos:** Precondición de ordenamiento.
- **2. Búsqueda Lineal:** Recorrido secuencial y evaluación de textos para filtros simples y compuestos.
- **3. Búsqueda por Transformación de Claves:** Integración con la tabla hash del almacén para consulta.